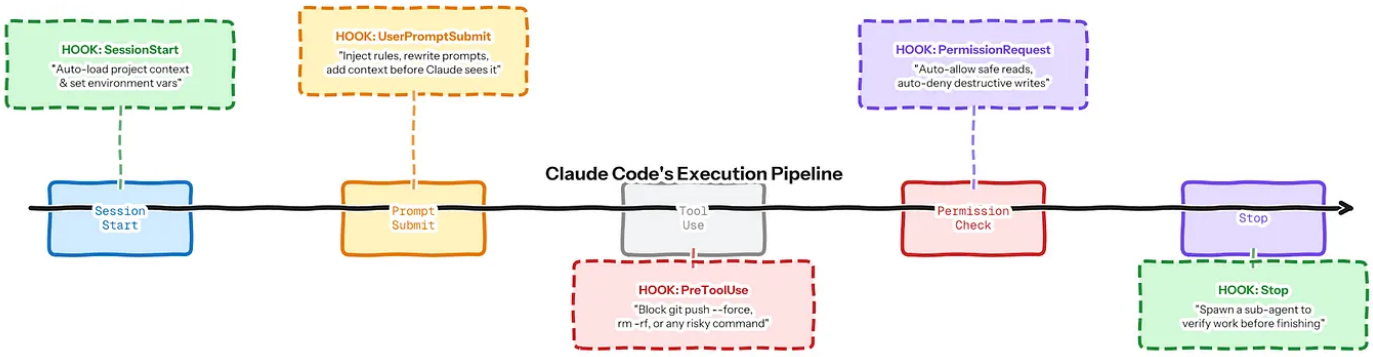
Agent Hooks Are Claude Code's Most Powerful Feature (and Almost Nobody Uses Them)

Vikas Sah Follow 9 min read · Mar 15, 2026

Move beyond 'lint on save.' Here are 7 production hooks — with code — that changed how I build with Claude.



Your code runs at every stage — intercepting, blocking, enriching, verifying.

20+ lifecycle events. 4 handler types. Most people use zero.

Last month, Claude deleted a production config file during a refactor. Technically, I approved the action — I clicked "Yes" without reading the full command. If you've used an AI coding assistant for more than a week, you know the feeling: the permission prompts come so fast that you start approving on autopilot.

That incident cost me three hours of debugging. But it also led me to discover what I now consider Claude Code's most overlooked feature: **hooks.**

If you've heard of hooks at all, you probably know them as "the thing that auto-formats your code after Claude writes it." That's like describing a smartphone as "a device that makes phone calls." Technically true. Wildly incomplete.

Claude Code's hook system has over 20 lifecycle events, 4 handler types, and an architecture that lets you spawn entirely separate AI agents to verify Claude's work before it's delivered. Most developers have never touched any of this. Most *non-developers* don't even know it exists.

This article will change that. Whether you write code or just work alongside people who do, understanding hooks will change how you think about governing AI systems. Here are 7 production hooks, with code, that transformed how I build.

First, the 30-Second Mental Model

Here's the core idea in plain English: **Claude is probabilistic. Hooks are deterministic.**

When you ask Claude to do something, it uses its best judgment. Most of the time, that judgment is excellent. But "most of the time" isn't good enough when the action is "delete a file" or "push code to production." Hooks are hard rules that fire automatically at specific moments in Claude's workflow. Claude doesn't get to override them. They're not suggestions — they're guardrails.

Think of it like this: Claude is a brilliant new hire. Hooks are the company policies that even brilliant hires must follow.

There are four types of hooks, and they form a natural ladder from simple to sophisticated:

1. **Command hooks** run a shell script. Fast, free, and deterministic. Think: "block any command containing rm -rf."
2. **HTTP hooks** call a remote endpoint. Useful for team-wide enforcement — one server, every developer's Claude respects the same rules.
3. **Prompt hooks** ask a fast AI model a yes/no question. Use these when the check requires judgment: "Does this code change match the intent of the task?"
4. **Agent hooks** spawn an entirely separate AI subagent that can read files, search code, and make multi-step decisions. This is the endgame — a

second AI that checks the first AI's work.

The decision framework is simple: **start with commands. Graduate to prompts when you need judgment. Use agents when you need investigation.**

Figure 1: The hook type decision ladder — more capability costs more, so start simple.

The 7 Hooks That Changed How I Build

Hook 1: The Security Gate

> Type: Command · Event: PreToolUse · Cost: Free

The problem: Claude can run any shell command if you approve it. When you're deep in a refactoring session and permission prompts are flying, it's easy to approve a destructive command by accident — rm -rf, git push — force, docker system prune. One misclick can ruin your afternoon.

The hook: A script that intercepts every shell command before it runs. It checks against a blocklist of dangerous patterns. If there's a match, it hard-blocks the action — Claude can't proceed, and you get a clear error explaining why.

```
#!/bin/bash
# .claude/hooks/security-gate.sh
COMMAND=$(cat | jq -r '.tool_input.command')

BLOCKED="rm -rf|git push.*--force|git reset --hard|drop table"
if echo "$COMMAND" | grep -qiE "$BLOCKED"; then
  echo "Blocked: destructive command detected" >&2
  exit 2  # EXIT 2 = HARD BLOCK
fi
exit 0    # EXIT 0 = PROCEED
```

Why it matters: Exit code 2 is the most important number in Claude Code. It's the only way to truly block an action. Exit code 1 just logs a warning and lets Claude proceed. If you build security hooks and forget the distinction, you have logging, not enforcement.

Hook 2: The Context Injector

> Type: Command · Event: UserPromptSubmit · Cost: Free

The problem: You type "fix the auth bug" and Claude starts working — but it doesn't know that your team migrated from JWT tokens to session-based auth last week, that there's an open PR touching the same files, or that your test suite has been flaky since Tuesday.

The hook: Every time you submit a prompt, this hook silently appends context — recent git history, open branches, and project-specific notes. Claude receives your prompt plus this background, like a colleague who's been sitting next to you all week.

```
#!/bin/bash
# .claude/hooks/context-injector.sh
CONTEXT=$(cat <<EOF
Recent commits: $(git log --oneline -5 2>/dev/null)
Current branch: $(git branch --show-current 2>/dev/null)
Open PRs: $(gh pr list --limit 3 --json title -q '.[].title' 2>/dev/null)
EOF
)
jq -n --arg ctx "$CONTEXT" '{
  hookSpecificOutput: {
    hookEventName: "UserPromptSubmit",
    additionalContext: $ctx
  }
}'
```

Why it matters: This is the hook that made Claude feel like a teammate instead of a stranger. For non-technical leaders, the takeaway is bigger: *context is the single biggest lever for AI quality.* When your AI assistant knows what happened yesterday, it makes dramatically better decisions today.

Hook 3: The Auto-Formatter

> Type: Command · Event: PostToolUse · Cost: Free

This is the hook that 90% of tutorials cover: run a code formatter after Claude writes a file. It works. It's useful. It's also table stakes.

I mention it for completeness, but I'm not going to pretend it's a revelation. The next four hooks are where things get genuinely interesting — because they cross the line from *utility* into *intelligence.*

Hook 4: The Premature Stop Detector

> Type: Prompt · Event: Stop · Cost: Minimal (single LLM call)

The problem: You ask Claude to do five things. It does three, says "Done!" and stops. This happens more often than you'd think, especially on complex tasks. AI models have a natural tendency to declare victory early.

The hook: When Claude tries to stop, a fast, inexpensive AI model reviews the conversation and checks: did Claude actually complete everything that

was requested? If not, it pushes Claude back to keep working.

```
// In .claude/settings.json
"hooks": {
  "Stop": [{
    "hooks": [{
      "type": "prompt",
      "prompt": "Review the conversation. The user asked
        Claude to complete a task. Did Claude finish ALL
        requested items? Return ok:false if anything is
        incomplete. $ARGUMENTS",
      "model": "claude-3-5-haiku-20241022"
    }]
  }]
}
```

Why it matters: This is your QA engineer who never sleeps. For managers and product leaders, this hook addresses one of the biggest risks of AI-assisted work: *the confident incomplete.* AI doesn't say "I'm not sure." It says "Done!" — and this hook catches the lie.

Hook 5: The Smart Permission Filter

> Type: Command · Event: PermissionRequest · Cost: Free

The problem: Claude asks permission 30 times during a refactor. Reading a file? Permission. Running a test? Permission. Checking git status? Permission. After the tenth prompt, you start clicking "Yes" without reading. That's exactly when the dangerous action slips through.

The hook: Auto-approve safe actions (reading files, running tests, searching code) while forcing manual review for risky ones (writing files, deleting anything, network calls). Fewer prompts means more attention for the prompts that matter.

```
#!/bin/bash
# .claude/hooks/smart-permissions.sh
TOOL=$(cat | jq -r '.tool_name')
SAFE="Read|Glob|Grep|Bash"  # read-only tools

if echo "$TOOL" | grep -qE "$SAFE"; then
  jq -n '{ hookSpecificOutput: {
    hookEventName: "PermissionRequest",
    decision: { behavior: "allow" }
  }}'
else
  exit 0  # Fall through to manual approval
fi
```

Why it matters: This is counterintuitive. More permission prompts don't make you safer — they make you *less* safe, because they exhaust your attention. The best security systems reduce noise so humans can focus on genuine decisions. This hook does exactly that.

Hook 6: The Verification Agent

> Type: Agent · Event: Stop · Cost: Higher (multi-turn subagent with tools)

The problem: Claude finishes a complex task. The code looks right. But

does it actually work? Do the tests pass? Are there side effects in other files?

The hook: When Claude finishes, a completely separate AI agent wakes up. This second agent can read files, search the codebase, and run multiple steps of verification. It reviews what Claude did, checks for regressions, and only approves the output if everything looks right.

```
"Stop": [{
  "hooks": [{
    "type": "agent",
    "prompt": "You are a code reviewer. Examine the
      changes made in this session. Check: (1) Do the
      changes match the original intent? (2) Are there
      unintended side effects? (3) Are tests passing?
      Return ok:false with a reason if ANY check fails.
      $ARGUMENTS",
    "model": "claude-3-5-haiku-20241022",
    "timeout": 120
  }]
}]
```

Why it matters: This is the hook that fundamentally changed how I think about AI-assisted work. It's not a human checking AI. It's *AI checking AI,* with full access to read files and run tools. For anyone managing a team that uses AI: this pattern — having a separate agent verify the primary agent's output — is the most practical form of AI governance available today. It works now, it's affordable, and it catches real bugs.

Hook 7: The Session Bootstrapper

> Type: Command · Event: SessionStart · Cost: Free

The problem: Every time you start a new Claude session, it's a blank slate. No memory of what you were working on, what branch you're on, or what's broken in CI.

The hook: A SessionStart hook that automatically gathers your project's current state — active branch, recent commits, CI status, open pull requests — and feeds it to Claude the moment a session begins. Every conversation starts warm instead of cold. It's the difference between briefing a contractor who just walked in the door versus continuing a conversation with a colleague who's been working alongside you all week.

The Bigger Picture: Hooks as a Governance System

Step back from the individual hooks and a larger pattern emerges. These seven hooks form a **layered governance system** — and the layers mirror how organizations govern human workers.

Automated compliance checks come first — no human in the loop, just clear rules. That's your command hooks. When the decision requires judgment, a single reviewer weighs in quickly. That's your prompt hooks. For high-stakes decisions, a full investigation is warranted. That's your agent hooks, spawning a subagent with full codebase access.

Figure 2: The 7 hooks mapped to Claude's lifecycle. Each fires at a specific point in the pipeline.

The crucial architectural insight is this: hooks operate *outside* Claude's context window. They're not instructions that Claude might creatively reinterpret. They're hard gates. When a command hook exits with code 2, the action is blocked — period. Claude doesn't get a vote.

This distinction matters for everyone, not just developers. As AI agents become more autonomous — making decisions, executing workflows, operating across multiple systems — the question of governance becomes critical. Hooks offer a practical answer: **let the AI be creative within defined boundaries, and enforce those boundaries with deterministic systems the AI cannot override.**

Strategic Takeaways

- **Hooks are governance, not automation.** The real value isn't running a

code formatter. It's building a system where AI operates within defined boundaries without constant human supervision. That's relevant whether you're a developer, a team lead, or a CTO.

- **The command → prompt → agent ladder is your migration path.** Start every hook as a command. Only graduate to prompt or agent hooks when the check genuinely requires reasoning or file access. Agent hooks cost tokens — use them where judgment matters, not where a shell script would do.
- **Exit code 2 is the most important number in Claude Code.** It's the only hard block. Everything else is advisory. If you're building security hooks and forget this distinction, you have logging, not enforcement.
- **"AI checking AI" is the most practical governance pattern available today.** Agent hooks prove the concept: a second AI with full codebase access verifying the first AI's work. This isn't theoretical. It works now, it costs pennies, and it catches real bugs. Every team using AI assistants should adopt this pattern.
- **The teams building hook libraries now will have a compounding advantage.** Every hook is reusable across projects, shareable via plugins, and versioned in Git. A year from now, the teams with mature hook libraries will be governing AI with the sophistication that their competitors are still trying to figure out.

Claude is brilliant. But brilliance without guardrails is a liability. Hooks are the guardrails.

Hit the *clap* if you found the article helpful!

Claude | Claude Code | Anthropic | Hooks | Agentic Ai

Written by Vikas Sah

181 followers · 43 following

Follow